HW 11

1. Bisection method, Interval on $[2,3]$ absolute error $= \frac{b-a}{2^n}$

$|x_k - x^*| \leq \epsilon \Rightarrow k \geq \log_2\left(\frac{b-a}{\epsilon}\right) - 1$ ← n = # iters before convergence

online sources don't include the -1 term

$= \log_2\left(\frac{1}{10^{-6}}\right) - 1$

$= \log_2(1) - \log_2(10^{-6}) - 1$

$= 0 + 6\log_2(10) - 1$

$= 18.932 \approx$ ~~19~~ iterations

relative error $= \frac{\epsilon}{|x^*|} = \frac{1/2^n}{|x^*|} \leq 10^{-6}$ 20 iterations

let's assume the worst case to guarantee the condition for any root in the interval $[2,3]$, which is when $|x^*|$ is small, so $|x^*| = \min(2,3) = 2$, making relative error the largest

$\frac{1/2^n}{2} \leq 10^{-6} \Rightarrow \frac{1}{2^n} \leq 2\cdot 10^{-6} \Rightarrow 2^n \geq \frac{10^6}{2} \Rightarrow 2^n \geq 5\cdot 10^5 \Rightarrow n \geq \log_2(5\cdot 10^5)$

$\Rightarrow n \geq \log_2(5) + \log(10^5) \Rightarrow n \geq \log_2(5) + 5\log(10) \approx 18.932$

$\approx$ 19 iterations

With relative error, the difference is one fewer iteration because dividing by the true root eliminates a factor of 2.

2. Newton's method on $f(x) = x^2 + 1$, take 5 steps from any starting point (not $x_0 = 0$)

$f'(x) = 2x$ $\qquad x_{k+1} = x_k - \frac{f(x_k)}{f'(x_k)}$

Let $x_0 = 2$

$x_1 = 2 - \frac{5}{4} = 0.75$

$x_2 = 0.75 - \frac{9/16}{3/2} \approx -0.2917$

$x_3 \approx -0.2917 - \frac{(-0.2917)^2 + 1}{2(-0.2917)} \approx 1.567$

$x_4 \approx 0.467$

$x_5 \approx -0.839$

$x^2 + 1$ has no real roots so it is diverging and alternating with no pattern to convergence. Hence, Newton's method requires a real root to exist nearby to converge to. Otherwise it acts insensibly.

3. $f(x,y) = x + e^{xy}$

$$H(x) = \begin{bmatrix} \frac{\partial^2 f}{\partial x^2} & \frac{\partial^2 f}{\partial x \partial y} \\ \frac{\partial^2 f}{\partial y \partial x} & \frac{\partial^2 f}{\partial y^2} \end{bmatrix} = \begin{bmatrix} y^2 e^{xy} & xye^{xy} + e^{xy} \\ xye^{xy} + e^{xy} & x^2 e^{xy} \end{bmatrix} = e^{xy} \begin{bmatrix} y^2 & xy+1 \\ xy+1 & x^2 \end{bmatrix}$$

$$\frac{\partial f}{\partial x} = 1 + ye^{xy} \qquad \frac{\partial f}{\partial y} = xe^{xy}$$

$$\frac{\partial^2 f}{\partial x \partial y} = xye^{xy} + e^{xy} \qquad \frac{\partial^2 f}{\partial y \partial x} = xye^{xy} + e^{xy}$$

$$\frac{\partial^2 f}{\partial x^2} = y^2 e^{xy} \qquad \frac{\partial^2 f}{\partial y^2} = x^2 e^{xy}$$

4. Let $A \in \mathbb{R}^{m \times n}$, $b \in \mathbb{R}^m$

a) $A = \begin{bmatrix} | & | & & | \\ a_1 & a_2 & \cdots & a_n \\ | & | & & | \end{bmatrix}$ (m rows, n columns)

Show $x^T A^T A x = \sum_{i,j} (a_i^T a_j) x_i x_j$ for any $x \in \mathbb{R}^n$

$$x^T A^T A x = (x_1 \cdots x_n) \begin{pmatrix} - a_1^T - \\ \vdots \\ - a_n^T - \end{pmatrix} \begin{pmatrix} | & & | \\ a_1 & \cdots & a_n \\ | & & | \end{pmatrix} \begin{pmatrix} x_1 \\ \vdots \\ x_n \end{pmatrix} = \begin{pmatrix} - x_1 a_1^T - \\ \vdots \\ - x_n a_n^T - \end{pmatrix} \begin{pmatrix} | & & | \\ a_1 & \cdots & a_n \\ | & & | \end{pmatrix} \begin{pmatrix} x_1 \\ \vdots \\ x_n \end{pmatrix} = \sum_{1 \le i,j \le n} (a_i^T a_j) x_i x_j \quad \checkmark$$

$$(x^T A^T A x)_{ij} = x_i a_i^T a_j x_j = (a_i^T a_j) x_i x_j$$

x_i, x_j are constants $\in \mathbb{R}$

Also sufficient to show

$$x^T A^T A x = (Ax)^T (Ax) = \|Ax\|_2^2 = \langle Ax, Ax \rangle$$

$$= \left\langle \sum_{i=1}^n x_i a_i, \sum_{j=1}^n x_j a_j \right\rangle = \sum_{i=1}^n \sum_{j=1}^n x_i x_j \langle a_i, a_j \rangle = \sum_{1 \le i,j \le n} (a_i^T a_j) x_i x_j \quad \checkmark$$

(bilinear)

b) $f(x) = \frac{1}{2} \|Ax - b\|_2^2$

$$f(x) = \tfrac{1}{2}(Ax-b)^T(Ax-b) = \tfrac{1}{2}(x^T A^T - b^T)(Ax - b) = \tfrac{1}{2}(x^T A^T A x - x^T A^T b - b^T A x + b^T b) = \tfrac{1}{2}(x^T A^T A x - 2 b^T A x + b^T b)$$

(x^T: $1 \times n$, A^T: $n \times m$, b: $m \times 1$; $x^T A^T b$ is 1×1, $b^T A x$ is 1×1: scalars)

1st term ↳ $\frac{\partial}{\partial x_k} \frac{1}{2} \sum_{i,j} (a_i^T a_j) x_i x_j = \frac{1}{2} \sum_j (a_k^T a_j) x_j + \frac{1}{2} \sum_i (a_i^T a_k) x_i = \sum_{j=1}^n (a_k^T a_j) x_j = A^T A x$

($\frac{\partial}{\partial x_i}$: $i = k$; $\frac{\partial}{\partial x_j}$: $j = k$)

$a_k^T a_j = a_j^T a_k$ dot products are symmetric

2nd term ↳ $\frac{\partial}{\partial x}(-b^T A x) = \frac{\partial}{\partial x}(-(A^T b)^T x) = -A^T b$

3rd term ↳ $\frac{\partial}{\partial x} \frac{1}{2} b^T b = 0$

$$\nabla f(x) = A^T A x - A^T b = \boxed{A^T(Ax - b)}$$

c) why x^* in $A^T A x^* = A^T b$ is a fixed point of GD on f ?

If x^* solves the equation $A^T A x^* = A^T b$, then the gradient of x^* is

$$\nabla f(x^*) = A^T(Ax^* - b) = A^T A x^* - A^T b = 0$$

So, the update step in gradient descent becomes

$$x_{k+1} = x^* - \alpha \cdot \nabla f(x^*) = x^* - \alpha \cdot 0 = x^*$$

Thus, x^* is a fixed point on f. ✓

5. f is at least 3 times differentiable

a) approx $f(x^*)$

Since x^* is a root, then $f(x^*)=0$, so

$$0 = f(x^*) \approx f(x_0) + f'(x_0)(x^* - x_0) + \frac{f''(x_0)}{2}(x^* - x_0)^2$$
$$= f(x_0) + (x^* - x_0)\left[f'(x_0) + \frac{f''(x_0)}{2}(x^* - x_0)\right]$$

$$\Rightarrow \quad x^* - x_0 = \frac{-f(x_0)}{f'(x_0) + \frac{1}{2}f''(x_0)(x^* - x_0)}$$

$$\Rightarrow \quad x^* \approx x_0 - \frac{f(x_0)}{f'(x_0) + \frac{1}{2}f''(x_0)(x^* - x_0)} \quad \checkmark$$

b) $x^* - x_0 \approx -\frac{f(x_0)}{f'(x_0)}$

$$x^* \approx x_0 - \frac{f(x_0)}{f'(x_0) + \frac{1}{2}f''(x_0)\left(-\frac{f(x_0)}{f'(x_0)}\right)} = x_0 - \frac{f(x_0)}{f'(x_0) - \frac{f''(x_0)f(x_0)}{2f'(x_0)}} = x_0 - \frac{f(x_0)f'(x_0)}{(f'(x_0))^2 - \frac{1}{2}f''(x_0)f(x_0)}$$

as a general iteration $x_0 \to x_k$, $x^* \to x_{k+1}$,

$$x_{k+1} = x_k - \frac{f(x_k)f'(x_k)}{(f'(x_k))^2 - \frac{1}{2}f''(x_k)f(x_k)} \quad \checkmark$$

c) The new iteration converged faster.

6. My perspective has changed in a more nuanced and cautious approach to computing / numerical analysis. From learning error bounds of floating point numbers and how certain matrix decompositions yield more efficient cost for solving linear systems, I emerged from this course with a more in-depth perspective in computing. I very much enjoyed the lectures on optimization as those topics haven't really cemented for me in my data science classes, but you were able to explain them simply. I would summarize the main takeaway as "there is always a more efficient method to solve linear systems."

```
f   = @(x) x^2 - 2;
df  = @(x) 2*x;
ddf = @(x) 2;

tol = 1e-12;
x0  = 10;

%% Newton's Method
x = x0;
fprintf("Newton's Method:\n");
fprintf("k\t xk\t\t\t f(xk)\n");
k = 0;
while abs(f(x)) > tol
    fprintf("%d\t %.15f\t %.2e\n", k, x, f(x));
    x = x - f(x)/df(x);
    k = k + 1;
end
fprintf("%d\t %.15f\t %.2e\n", k, x, f(x));
fprintf("Newton converged in %d iterations\n\n\n", k);

%% New Iteration
x = x0;
fprintf("New Iteration:\n");
fprintf("k\t xk\t\t\t f(xk)\n");
k = 0;
while abs(f(x)) > tol
    fprintf("%d\t %.15f\t %.2e\n", k, x, f(x));
    x = x - (f(x)*df(x)) / (df(x)^2 - 0.5*ddf(x)*f(x));
    k = k + 1;
end
fprintf("%d\t %.15f\t %.2e\n", k, x, f(x));
fprintf("New iteration converged in %d iterations\n", k);

% OUTPUT
% Newton's Method:
% k  xk      f(xk)
% 0  10.000000000000000  9.80e+01
% 1  5.100000000000000   2.40e+01
% 2  2.746078431372549   5.54e+00
% 3  1.737194874379598   1.02e+00
% 4  1.444238094866232   8.58e-02
% 5  1.414525655148738   8.83e-04
% 6  1.414213596802269   9.74e-08
% 7  1.414213562373095   8.88e-16
% Newton converged in 7 iterations
%
% New Iteration:
% k  xk      f(xk)
% 0  10.000000000000000  9.80e+01
% 1  3.509933774834437   1.03e+01
% 2  1.650475173253008   7.24e-01
% 3  1.415510038078371   3.67e-03
% 4  1.414213562645118   7.69e-10
% 5  1.414213562373095   4.44e-16
% New iteration converged in 5 iterations
```